Multi-Threading

Multi Tasking

Executing several tasks simultaneously is the concept of multitasking. There are two types of multitasking's.

1. Process Based
2. Thread Based

Process based multitasking:

- Here several tasks which are executed simultaneously are separate independent **processes of OS**
- Ex. Creating a word file while playing songs in media player.

Thread based multitasking:

- Here several tasks which are executed simultaneously are separate independent part of **same single program**.
- Each independent part is called a **"Thread"**
- In multithreading on 10% of the work the programmer is required to do and 90% of the work will be down by java API.
- The main important application areas of multithreading are:
    1. To implement multimedia graphics.
    2. To develop video games etc.
    3. To develop web and application servers.

Ways to define a new Thread

1. By Extending Thread class

```
class MyThread1 extends Thread{
    public void run(){
        for(int i=0;i<10;i++){
            System.out.println("child thread");
        }
    }
}
public class MultiThreadingUsingThreadClass {
    public static void main(String[] args) {
        MyThread1 t1 = new MyThread1();
        t1.start();
    }
}
```

- Here MyThread1 **extends** Thread class and **overrides** the run() method.
- It is **important to override run()** because the Thread class **run() is Empty**.
- By overriding it we provide our task to be done.
- **t1.start()** creates a **new thread** performing the task in run().

2. By Implementing Runnable Interface

```
class MyRunnable implements Runnable{
    @Override
    public void run() {
        for(int i=0;i<10;i++){
            System.out.println("hello STAR");
        }
    }
}
public class MultiThreadingUsingRunnableInterface {
    public static void main(String[] args) {
        MyRunnable task = new MyRunnable();
        Thread t1 = new Thread(task);
        //task is object with runnable implementation
        t1.start();
    }
}
```

- Here **MyRunnable** class is **implementing Runnable** interface and overrides run().
- It is important to override run(), because Runnable interface has **abstract** run().
- Created new **Thread object t1** with **Runnable implementation passed in constructor**.
- And start() method is called on that thread object.

3. By using Lambda function

```
Thread t1 = new Thread(()->{
    for(int i=0;i<10;i++){
        System.out.println(i);
    }
});
//lambda is implementation of Runnable interface
t1.start();
```

- Here we **used lambda** to implement the Runnable and **passed it in constructor** of Thread.
- As Runnable is **Functional Interface** we can **use Lamba** function here.
- Started a new Thread by using start() method.

Thread Class

Thread Scheduler:

- If multiple Threads are waiting to execute, then which Thread will execute 1st is decided by "Thread Scheduler" which is part of JVM. We cannot expect the actual behaviour of Thread Scheduler.

Difference between t.start() and t.run() methods:

- **t.start()**
  - Creates a **new thread**.
  - Calls the run() method **in that new thread**.
  - Executes **concurrently** with the main thread.
- **t.run()**
  - Just a normal method call.
  - Runs in the **same thread** (e.g., the main thread).
  - **No new** thread is created

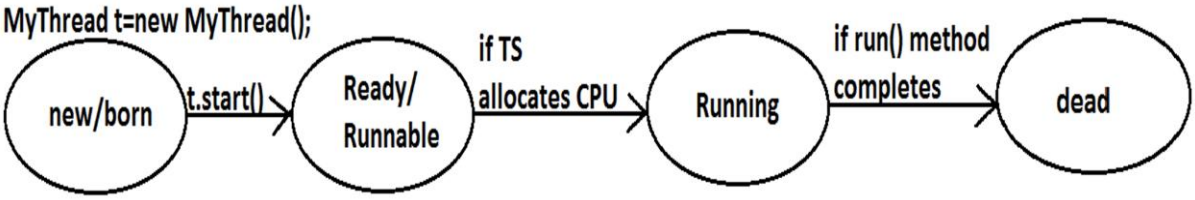
importance of Thread class start() method:

- For every new Thread the required mandatory activities like registering the Thread with Thread Scheduler will taken care by **Thread class start() method**.
- programmer needs to define the job of the Thread **inside run() method**.
- That is start() method acts as best assistant to the programmer.
- Without executing Thread class start() method there is no chance of starting a new Thread in java.
- We must override run() method because Thread class run() method has empty implementation and hence we won't get any output without overriding.

Thread class Constructors

1. Thread t1 = new Thread();
2. Thread t1 = new Thread(Runnable r);
3. Thread t1 = new Thread(String name);
4. Thread t1 = new Thread(Runnable r, String name);
5. Thread t1 = new Thread(ThreadGroup g, String name);
6. Thread t1 = new Thread(ThreadGroup g, Runnable r);
7. Thread t1 = new Thread(ThreadGroup g, Runnable r, String name);
8. Thread t1 = new Thread(ThreadGroup g,Runnable r,String name,long stackSize);

Life Cycle of THREAD:



- Once we created a Thread object then the Thread is said to be in new state or born state.
- Once we call start() method then the Thread will be entered into Ready or Runnable state.
- If Thread Scheduler allocates CPU then the Thread will be entered into running state.
- Once run() method completes then the Thread will entered into dead state.

(NOTE: After starting a Thread we are not allowed to restart the same Thread once again otherwise we will get runtime exception saying "IllegalThreadStateException".)

Best approach to define a Thread:

- Among the 3 ways of defining a Thread, **implements Runnable** approach is always recommended.
- In the 1st approach we **need to extend Thread class** and after that we **cannot extend any other**. Thus, we are **missing benefits of inheritance** here.
- But in the 2nd approach **while implementing Runnable** interface **we can extend some other class** also. Hence implements Runnable mechanism is recommended to define a Thread.

Thread Methods:

1) Thread.currentThread()

❖ Static method, Gives the object referencing to the current executing thread.

2) .setName(String name)

❖ Used to explicitly name the thread.

3) .getName()

❖ Used to get the name of the thread.

4) .setPriority(int Priority)

❖ Used to set the priority of the thread explicitely.

5) .getPriority()

❖ Used to get the priority of the thread.

(Note: The valid range of Thread priorities is 1 to 10, **1 is lowest** and **10 is highest** . thread class have following constatnts i.e**. Thread.MIN_PRIORITY**(1), **Thread.NORM_PRIORITY**(5) and **Thread.MAX_PRIORITY**(10).

The Methods to Prevent a Thread from Execution:

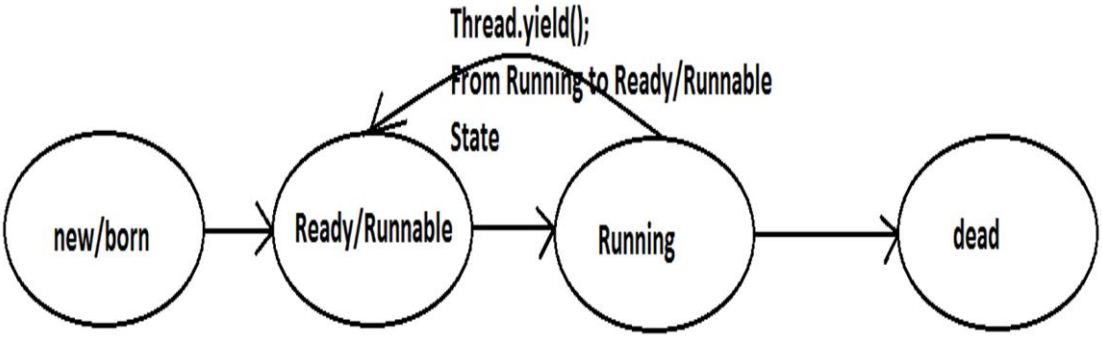
- We can prevent(stop) a Thread execution by using the following methods.

1. yield()
2. join()
3. sleep()

Note: join() and sleep() throws InteruptedException which is checked so need to handle it with try-catch

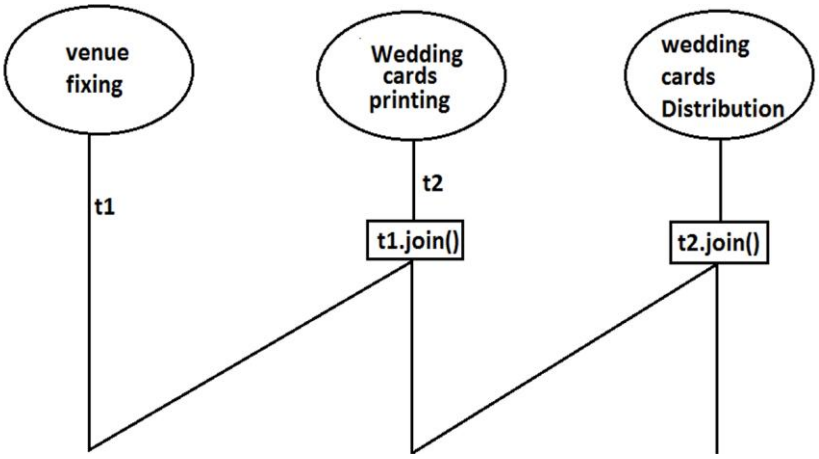
yield():

- it stops the current executing thread and gives chance of execution to other waiting threads with same priority of it.
- If there is no waiting thread with same priority current thread will continue to execute
- If multiple threads with same priority are waiting then we can't expect which will be executing first
- public static native void yield();



Join():

- if thread wants to wait until the other threads can execute then we should go for join() method
- If a Thread t1 executes t2.join() then t1 should go for waiting state until completing t2.



(Note : If main thread calls join() on child thread object and child thread called join() on main thread object then both threads will wait for each other forever and the program will be hanged(like deadlock))

Sleep(*long ms*):

- If thread don't want to perform any operation for a specific amount of time then we use sleep method

Interrupting A Thread

- We can interrupt only sleeping or waiting Thread.
- If a Thread never goes to sleep or wait then it can't be interrupted.
- We can interrupt a thread by using interrupt() method of thread class.

Synchronization:

- Synchronized is the keyword applicable for methods and blocks but not for classes and variables.
- If a method or block declared as the synchronized then at a time only one Thread is allowed to execute that method or block on the given object.
- The main advantage of synchronized keyword is we can resolve data inconsistency problems.
- But the main disadvantage of synchronized keyword is it increases waiting time of the Thread and effects performance of the system
- It uses locking mechanism, where only one object with acquired lock can access the synchronised method at a time.
- So at a time only one thread is given access to synchronized methods of the class.
- Synchronized block is used when we want very few lines synchronized.
- **Statements present inside synchronized block are called synchronized statements.**

Conclusion: If multiple threads are operating on multiple objects then there is no impact of Synchronization. If multiple threads are operating on same java objects then synchronized concept is required(applicable).

Locking types

Object level lock

- When we are using synchronized to make an instance method synchronized then the Thread acquires object level lock.
- One thread is allowed to run that code for given instance.
- For different instance another thread can run simultaneously.

```
void printHello(){
    System.out.println("Hello STAR");
}
```

- You can have object level lock by using synchronized block.
- But synchronized block can be only inside **method or constructor or initializer block.**

```
void printHello(){
    synchronized(this){
        System.out.println("Hello STAR");
    }
}
```

Class level lock

- When we are using synchronized to make an static method synchronized then the Thread acquires class level lock.
- only one thread in entire JVM is allowed to run that code- regardless of umber of objects created

```
static synchronized void printHello(){
    System.out.println("Hello STAR");
}
```

- You can have class level lock by using synchronized block.
- But synchronized block can be only inside **method or constructor or initializer block**

```
void printHello(){
    synchronized(Hello.class){
        System.out.println("Hello STAR");
    }
}
```

NOTE: As both object level lock and class level lock are different, they do not block each other and never blocks normal method execution. So three threads can run object level locked, class level locked and normal methods simultaneously.

Inter Thread Communication

- Inter-Thread Communication (ITC) in Java means **two or more threads talking to each other** so that they can work **together** instead of working blindly in parallel.
- ITC helps threads **coordinate**, especially when one thread needs data from another thread.
- It solves producer consumer problem between threads related to data transfer.
- For ITC java provide three methods in Object class, so does every object.
    1. wait()
    2. notify()
    3. notifyAll()
- wait(): Makes the thread release the lock, adds it to the objects waiting pool, Thread stops running until someone notifies it. **Must be called inside synchronized block/method.**
- notify(): Wakes ONE thread from waiting pool and to run after it completes; **but** the awakened thread does NOT run immediately, it waits to regain the lock.
- notifyAll(): wakes all waiting threads to run at after it compltes, but only one will eventually get the lock first.

Note: wait(), notify(), notifyAll() must be used inside synchronized block/method

Dead lock

- If 2 Threads are waiting for each other forever(without end) such type of situation(infinite waiting) is called dead lock.
- There are no resolution techniques for dead lock but several prevention(avoidance) techniques are possible.
- Synchronized keyword is the cause for deadlock hence whenever we are using synchronized keyword we have to take special care.
- For Example see the java core codes project/Deadlock in intellij

Daemon Threads

- A **daemon thread** is a background thread that provides services to **non-daemon (user) threads**
- Examples:
    1. Garbage Collector (GC)
    2. Finalizer Thread
    3. Timer Background Task
    4. JVM housekeeping threads
- We can check whether the Thread is daemon or not by using isDaemon() method of Thread class.
- We can change daemon nature of a Thread by using setDaemon () method.
- But we can change daemon nature before starting Thread only.
- Default Nature : Main Thread is always non daemon and we can't change its daemon nature because it's already started at the beginning only.
- Main Thread is always non-daemon, and for other threads daemon nature is inherited from parent thread.
- After the termination of last non-daemon thread, all daemon threads are terminated

Deadlock vs Starvation:

- A long waiting of a Thread which never ends is called deadlock.
- A long waiting of a Thread which ends at certain point is called starvation.
- A low priority Thread has to wait until completing all high priority Threads.
- This long waiting of Thread which ends at certain point is called starvation.

RACE condition:

- A **race condition** happens when **multiple threads access and modify shared data at the same time**, and the final result depends on **which thread runs first**.
- Causes data inconsistency

Multi-Threading Enhancements

Concurrent locks

Problems with traditional synchronized key word

- If a thread releases the lock which waiting thread will get that lock is not in our control.
- There is no API to list all waiting threads for a lock.
- With synchronized we only have intrinsic (implicit) locks and limited control: no ability to check queue, fairness, try-lock with timeout, etc. It's also less flexible compared to Lock API.
- To overcome this we got ***java.util.concurrent.locks*** package in java 5.

Lock (Interface)

- A Lock Object is Similar to Implicit Lock acquired by a Thread to Execute synchronized Method OR synchronized Block.
- Lock Implementations Provide More Extensive Operations than Traditional Implicit Locks.
- Important methods of Lock Interface are
    1. void lock()
        - It Locks the Lock Object.
        - If Lock Object is Already Locked by Other Thread then it will wait until it is Unlocked.
    2. boolean tryLock()
        - To Acquire the Lock if it is Available.
        - If the Lock is Available then Thread Acquires the Lock and Returns true.
        - If the Lock Unavailable then this Method Returns false and Continue its Execution.
        - In this Case Thread is Never Blocked.
    3. boolean tryLock(Long time,TimeUnit unit)
        - To Acquire the Lock if it is Available.
        - If the Lock is Unavailable then Thread can Wait until specified Amount of Time.
        - Still if the Lock is Unavailable then Thread can Continue its Execution.
    4. void lockInterruptibly()
        - acquires the lock util current thread is interrupted
        - Acquires the Lock if it is Available and Returns Immediately.
        - If it is Unavailable then the Thread will wait, while waiting if it is Interrupted then it won't get the Lock.
    5. void unlock()
        - releases the lock

Reenterant Lock(Class)

- It implements Lock Interface and it is the Direct Child Class of an Object.
- Reentrant Means a Thread can acquires Same Lock Multiple Times without any Issue.
- Internally ReentrantLock Increments Threads Personal Count whenever we Call lock() and Decrements Counter whenever we Call unlock() and Lock will be Released whenever Count Reaches '0'.
- Constructors of reentrant lock
    1. ReentrantLock rl = new ReentrantLock()
    2. ReentrantLock rl = new ReentrantLock(boolean fairness)
        - If fairness is true then the longest waiting thread will acquire the lock first
        - If fairness is false then no guarantee of which tread will acquire the lock
        - By default it is false
- Important methods of RentrantLock class
    1. void lock()
    2. boolean tryLock()
    3. boolean tryLock(Long Time, TimeUnit Unit)
    4. void lockInterruptely()
    5. void unlock()
    6. int getHoldCount()
        - return number of holds on lock by current thread
    7. boolean isHeldByCurrentThread()
        - return true it the lock is only held by current thread.
    8. int getQueueLength()
        - Return the number of threads waiting for the lock
    9. protected Collection getQueuedThreads()
        - Return colletion containing the threads objects waiting for the lock.
    10. boolean hasQueuedThreads()
        - Return true if any thread is waiting for the lock
    11. boolean isLocked()
        - Return true if lock is acquired by any thread
    12. boolean isFair()
        - Return true if lock fairness is set to true
    13. protected Thread getOwner()
        - Return the thread which acquired the lock.

Conditions

- A **Condition** in Java is a synchronization construct associated with a **Lock** that allows threads to wait until a specific condition becomes true while temporarily releasing the lock.
- Following methods
  - **.await()**
  - **.signal()**
  - **.signalAll()**
- When a thread calls **await()**, it **releases the lock**, enters the **Condition's waiting queue**, and goes into the **WAITING state**.
- When **signal()** is called, **one waiting thread** is moved from the condition queue to the lock's queue and will resume execution **after re-acquiring the lock**.
- When **signalAll()** is called, **all waiting threads** are moved to the lock's queue and compete to re-acquire the lock.
- await(), signal(), and signalAll() **must be called while holding the associated lock**, otherwise IllegalMonitorStateException is thrown.

Note : Conditions gives better control over the synchronization with coordination

Thread pools

- Creating a New Thread for Every Job May Create Performance and Memory Problems.
- To Overcome this we should go for Thread Pool Concept.
- Thread Pool is a Pool of Already Created Threads Ready to do Our Job.
- Thread Pool Framework is Also Known as Executor Framework.
- We can Create a Thread Pool as follows

```
• ExecutorService service = Executors.newFixedThreadPool(3);
  //created thread pool of 3 threads
```

- We can Submit a Runnable Job by using submit().
- We can ShutdownExecutiorService by using shutdown().

```
• ExecutorService pool = Executors.newFixedThreadPool(5);
  //created thread pool of 3 threads
  pool.execute(()->{
      for(int i=0;i<50;i++){
          System.out.println("HI Madam");
      }
  });
  //here task is Runnable implemented by lambda
  pool.execute(()->{
      for(int i=0;i<50;i++){
          System.out.println("Hi sir");
      }
  });
  pool.shutdown();
```

Note: Usually we can Use ThreadPool Concept to Implement Servers (Web Servers And Application Servers)

Callable and Future

- In the Case of Runnable Job Thread won't Return anything.
- If a Thread is required to Return Some Result after Execution then we should go for Callable.
- Callable Interface contains Only One Method public Object call() throws Exception.
- If we Submit a Callable Object to Executor then the Framework Returns an Object of Type java.util.concurrent.Future
- The Future Object can be Used to Retrieve the Result from Callable Job.

```
• ExecutorService executor = Executors.newFixedThreadPool(4);

  Callable<String> task = ()->{
      System.out.println("doing the work.....");
      return "returned from task";
  };

  Future<String> future=executor.submit(task);
  //executor takes Runnable, Callable

  executor.shutdown();

  System.out.println(future.get());
  //future.get() to get the result of callable
  //check if task is done future.isDone()
```

Execute run the task without returning anything. Used for Runnable

Submit returns Future<>. Used for Callable